

Report secondo progetto di Metodi del calcolo scientifico

Daniele Besozzi 894998
Andrea Consonni 900116
Matteo Cervini 902225

<https://github.com/CerviniMatteo/progetto2-DCT-Compression>

4 giugno 2026

Indice

1	Prima parte: implementazione e benchmark DCT2	2
1.1	Metodologia di misura delle performance	2
1.2	Risultati	3
2	Seconda parte: compressione immagini	4
2.1	Test su immagini	4
2.1.1	Tony Pitony	4
2.1.2	Lena	8
2.1.3	Scacchiera	9
3	Struttura del progetto	11
3.1	Implementazione della DCT2	12
3.2	Gestione dell'esecuzione	13
3.2.1	GUI	14

Capitolo 1

Prima parte: implementazione e benchmark DCT2

Per l'implementazione della DCT2 abbiamo scelto il linguaggio di programmazione Java, mettendo a confronto la nostra implementazione con l'implementazione fornita dalla libreria `JTransform` [1]. Per effettuare le operazioni di algebra lineare richieste in modo semplice ed ottimale, abbiamo scelto di utilizzare la libreria `Efficient Java Matrix Library`(ejml)[2].

1.1 Metodologia di misura delle performance

Per misurare le performance della nostra implementazione rispetto a quella della libreria, abbiamo generato in maniera causale matrici di dimensioni pari a 2^i , $i = 3, \dots, 13$. Le entrate di questa matrice sono generate in maniera casuale in un intervallo di valori a virgola mobile compreso tra 0 e 100.

```
1  public static double[][] randomMatrix(int n) {
2
3      //Creates a square matrix(n x n)
4      double[][] m = new double[n][n];
5
6      //Double loop to fill the matrix
7      for (int i = 0; i < n; i++) {
8          for (int j = 0; j < n; j++) {
9              //Fill the i,j-th entry with a random double value between 0 and 100
10             m[i][j] = Math.random()*100;
11         }
12     }
13
14     return m;
15 }
```

Listing 1.1: Generazione di matrici casuali

Per eseguire le misurazioni, abbiamo utilizzato la libreria `Java Microbenchmark Harness`(JMH)[3], che ci ha permesso di eseguire i benchmark in modo affidabile e preciso, gestendo automaticamente le iterazioni di warmup e misurazione. Poiché Java esegue ottimizzazioni a runtime (JIT compilation), è necessario eseguire delle iterazioni di warmup prima delle misurazioni effettive. Durante la fase di warmup, il codice viene compilato e ottimizzato dalla JVM (Java Virtual Machine)[4]. Successivamente, eseguiamo molteplici run e calcoliamo la media dei tempi ottenuti da queste iterazioni, scartando i risultati della fase di warmup. Il codice di configurazione del benchmark è riportato nel listing 1.2.

```

1 public double run(Supplier<Supplier<?>> taskFactory) throws Exception {
2
3     Options opt = new OptionsBuilder()
4         // Format the JMH include regex with the benchmark class name.
5         // e.g. ".*BenchmarkRunner\\.run" matches any fully-qualified name ending with
6         // "BenchmarkRunner.run"
7         .include(String.format(
8             BenchmarkConstants.JMH_BENCHMARK_INCLUDE_TEMPLATE,
9             BenchmarkRunner.class.getSimpleName()))
10        .forks(0)
11        .warmupIterations(DEFAULT_WARMUP_ITERATIONS)
12        .measurementIterations(DEFAULT_MEASUREMENT_ITERATIONS)
13        .build();
14
15    Collection<RunResult> results;
16    try {
17        pendingFactory = taskFactory;
18        results = new Runner(opt).run();
19    } finally {
20        // Clear the shared factory after this run so it cannot be reused
21        // accidentally in later benchmarks and so captured objects can be
22        // garbage-collected.
23        pendingFactory = null;
24    }
25
26    if (results.isEmpty()) {
27        throw new CancellationException(BenchmarkConstants.BENCHMARK_ABORTED_NO_RESULTS);
28    }
29    return results.iterator().next().getPrimaryResult().getScore();
30 }

```

Listing 1.2: Configurazione del benchmark con JMH

1.2 Risultati

I risultati del benchmark sono riportati nella figura 1.1. Come si può evincere dal grafico, i tempi di

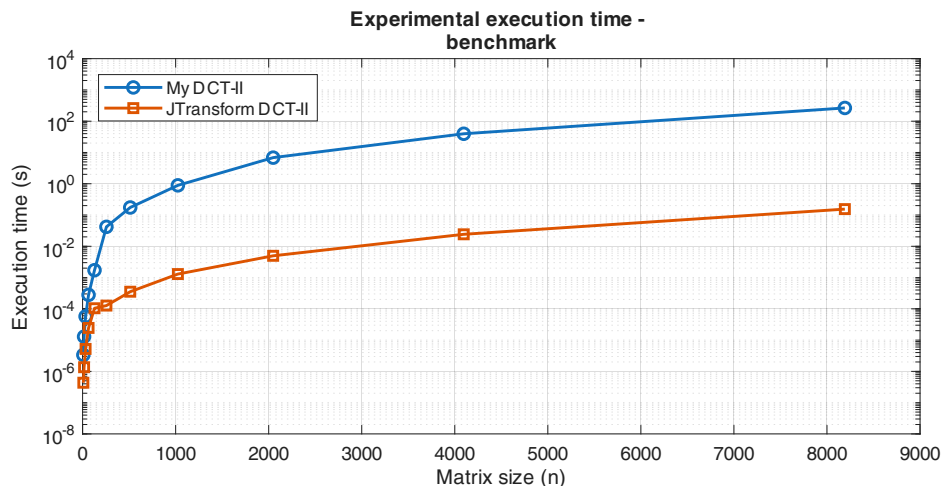


Figura 1.1: Risultati del benchmark tra la nostra implementazione della DCT2 e quella fornita da JTransform

esecuzione dell'implementazione fornita dalla libreria sono di gran lunga inferiori rispetto alla nostra implementazione. L'algoritmo implementato dalla libreria è nella versione fast (FFT) e implementa una variante dell'algoritmo dividi-et-impera per la FFT (split-radix e mixed-radix) ottimizzato per sistemi multiprocessore [5]. Ciò comporta un miglioramento significativo delle prestazioni dato sia dall'algoritmo utilizzato, sia dalla capacità della libreria di sfruttare le capacità di multithreading del processore.

Capitolo 2

Seconda parte: compressione immagini

2.1 Test su immagini

Per provare la nostra implementazione della compressione tramite DCT2, abbiamo scelto tre immagini: una di Tony Pitony, una di Lena (immagine classica per l'immagine processing) e una scacchiera di 40×40 pixel. Per tutte le immagini, abbiamo applicato la DCT2 con diversi valori di f (la dimensione del blocco) e d (il coefficiente del taglio). Di seguito sono riportati i risultati ottenuti per ciascuna immagine, le quali sono anche disponibili in formato .bmp al [link](#).

2.1.1 Tony Pitony

Per quanto riguarda l'immagine in figura 2.1 abbiamo provato a modificare la dimensione dei blocchi f e il coefficiente del taglio d . I valori di f che abbiamo scelto sono stati 8, 16, 32, 64 e 128, mentre i valori di d sono stati rispettivamente 4, 8, 16, 32 e 64, ovvero la metà della dimensione del blocco. Fare benchmark aumentando la dimensione dei blocchi può risultare interessante per questa immagine perché contiene sia aree uniformi sia dettagli ad alta frequenza, come i bordi degli occhiali e la texture della camicia. Blocchi più grandi migliorano generalmente la compressione nelle zone lisce, ma possono introdurre artefatti più evidenti e perdita di dettaglio nelle regioni ricche di dettagli.



Figura 2.1: Immagine di Tony Pitony non compressa



Figura 2.2: Immagine di Tony Pitony compressa con DCT2, con $f = 8$ e $d = 4$



Figura 2.3: Immagine di Tony Pitony compressa con DCT2, con $f = 16$ e $d = 8$



Figura 2.4: Immagine di Tony Pitony compressa con DCT2, con $f = 32$ e $d = 16$



Figura 2.5: Immagine di Tony Pitony compressa con DCT2, con $f = 64$ e $d = 32$



Figura 2.6: Immagine di Tony Pitony compressa con DCT2, con $f = 128$ e $d = 64$

All'aumentare della dimensione dei blocchi, possiamo vedere che la qualità dell'immagine compressa peggiora particolarmente nelle aree con molti dettagli. In particolare, si può anche notare l'introduzione di artefatti di tipo ringing nelle zone uniformi, come la finestra, che diventano più evidenti con blocchi più grandi. Un esempio di questi artefatti è mostrato in figura 2.7, dove si possono vedere delle bande scure e chiare intorno ai bordi, tipiche del fenomeno di ringing. L'effetto sulle nostre immagini è evidenziato in figura 2.8, dove si possono notare le bande scure attorno alla gamba e ai bordi della finestra.



Figura 2.7: Artefatti di tipo ringing



Figura 2.8: Zoom sull'immagine di Tony Pitony compressa con DCT2, con $f = 128$ e $d = 64$, evidenziando gli artefatti di tipo ringing

2.1.2 Lena

Per quanto riguarda l'immagine in figura 2.9 abbiamo provato a mantenere fissa la dimensione dei blocchi f a 8 e a modificare il coefficiente del taglio d con i valori 12, 8, 4, 2 e 0. In particolare, con $d = 0$ abbiamo eliminato tutti i coefficienti, ottenendo così un'immagine completamente nera. Si può osservare che, come ci si aspetta, all'aumentare del numero di coefficienti mantenuti, la qualità dell'immagine compressa migliora, con una maggiore fedeltà all'immagine originale. In particolare, con $d = 12$ e $d = 8$ l'immagine compressa è molto simile all'originale, mentre con $d = 4$ si notano perdite di dettagli e si manifesta il tipico effetto di jpeg dove sono ben visibili i blocchi di dimensione 8×8 .



Figura 2.9: Immagine di Lena non compressa



Figura 2.10: Immagine di Lena compressa con DCT2, con $f = 8$ e $d = 12$



Figura 2.11: Immagine di Lena compressa con DCT2, con $f = 8$ e $d = 8$



Figura 2.12: Immagine di Lena compressa con DCT2, con $f = 8$ e $d = 4$



Figura 2.13: Immagine di Lena compressa con DCT2, con $f = 8$ e $d = 2$

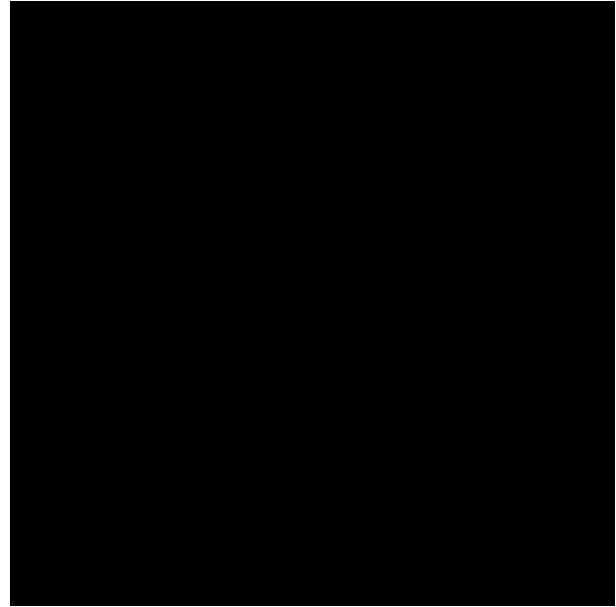


Figura 2.14: Immagine di Lena compressa con DCT2, con $f = 8$ e $d = 0$

2.1.3 Scacchiera

L'analisi dell'immagine in figura 2.15 è interessante in quanto permette di osservare il contributo del coefficiente della componente continua nella trasformata coseno di immagini. Dato che l'immagine è composta da aree uniformi di colore bianco e nero, la componente a frequenza 0 (ovvero il coefficiente della componente continua) è particolarmente importante per mantenere la struttura dell'immagine. Ogni quadrato della scacchiera è di dimensione 10×10 pixel. Scegliendo il parametro f in modo da far corrispondere i blocchi con le sezioni uniformi dei quadrati della scacchiera (ad esempio $f = 10$ e $f = 5$) e mantenendo solo il coefficiente della componente continua (ovvero $d = 1$), otteniamo un'immagine compressa che mantiene la struttura della scacchiera. Questo accade in quanto essendo i quadrati uniformi, la componente continua è sufficiente per rappresentarli. L'effetto della compressione eseguita con questi parametri è mostrato in figura 2.16 e in figura 2.17. Scegliendo invece $f = 20$, quindi prendendo un blocco che contiene due quadrati neri e due quadrati bianchi, e mantenendo solo il coefficiente della componente continua, otteniamo un'immagine compressa che perde completamente la struttura della scacchiera, risultando in un'immagine grigia uniforme, come mostrato in figura 2.18. Aumentando invece il numero di coefficienti mantenuti, ad esempio con $d = 10$ o $d = 20$, otteniamo un'immagine compressa che recupera parzialmente la struttura della scacchiera, ma con una qualità visiva inferiore rispetto ai casi con $f = 10$ o $f = 5$, come mostrato in figura 2.19 e in figura 2.20.

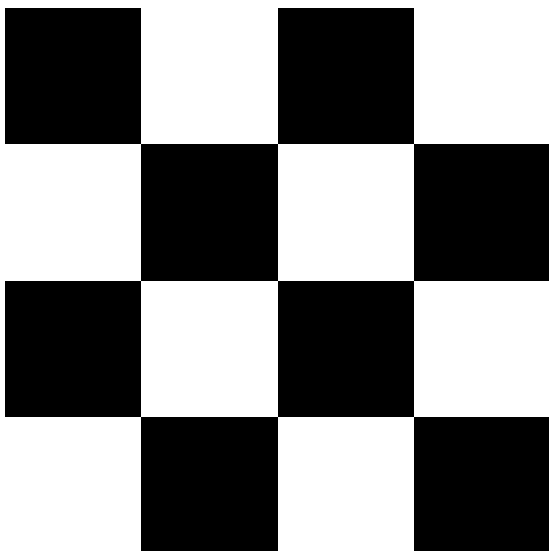


Figura 2.15: Scacchiera non compressa

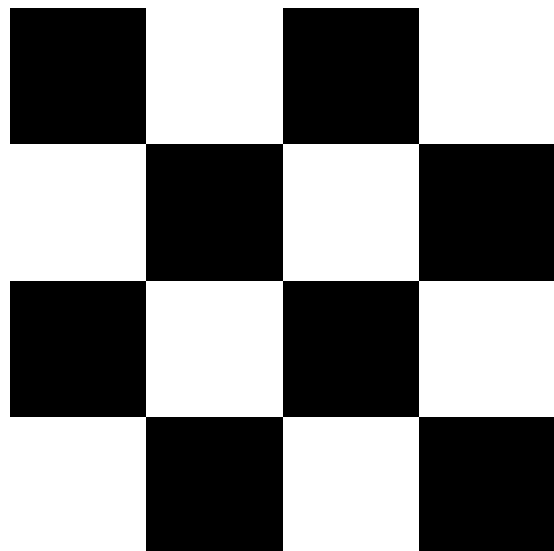


Figura 2.16: Scacchiera compressa con DCT2, con $f = 10$ e $d = 1$

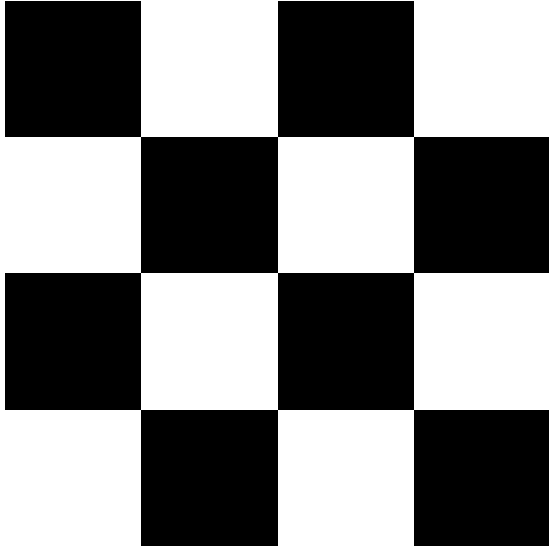


Figura 2.17: Scacchiera compressa con DCT2, con $f = 5$ e $d = 1$

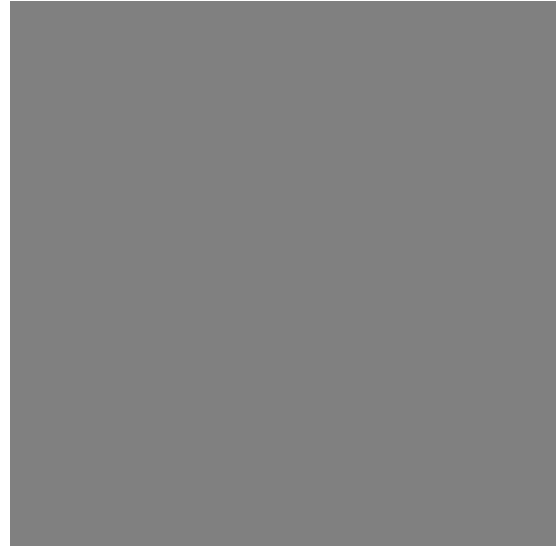


Figura 2.18: Scacchiera compressa con DCT2, con $f = 20$ e $d = 1$

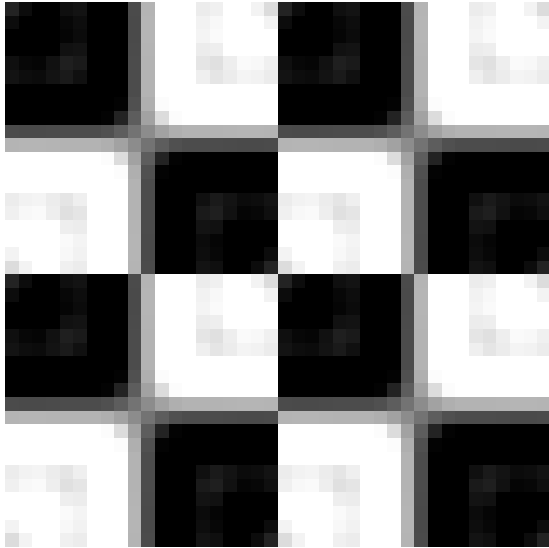


Figura 2.19: Scacchiera compressa con DCT2, con $f = 20$ e $d = 10$

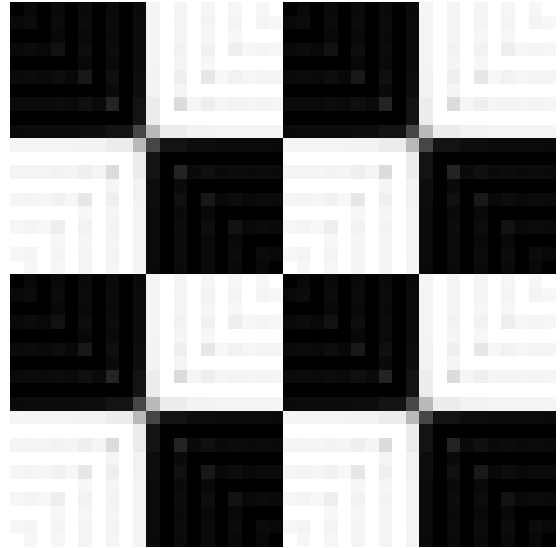


Figura 2.20: Scacchiera compressa con DCT2, con $f = 20$ e $d = 20$

Capitolo 3

Struttura del progetto

La struttura del progetto è organizzata in due macro package:

- **GUI**(Graphical User Interface), il quale si occupa unicamente di gestire l'interfaccia utente
- **backend**, che contiene :
 - l'implementazione della nostra DCT2
 - la configurazione dei benchmark
 - la logica per la compressione delle immagini (con la DCT2 nel formato fast offerta dalla libreria esterna **JTransform**)
 - la generazione dei risultati in formato CSV per i benchmark e immagini in formato BMP per la compressione delle immagini.

La figura 3.1 mostra la struttura del progetto.

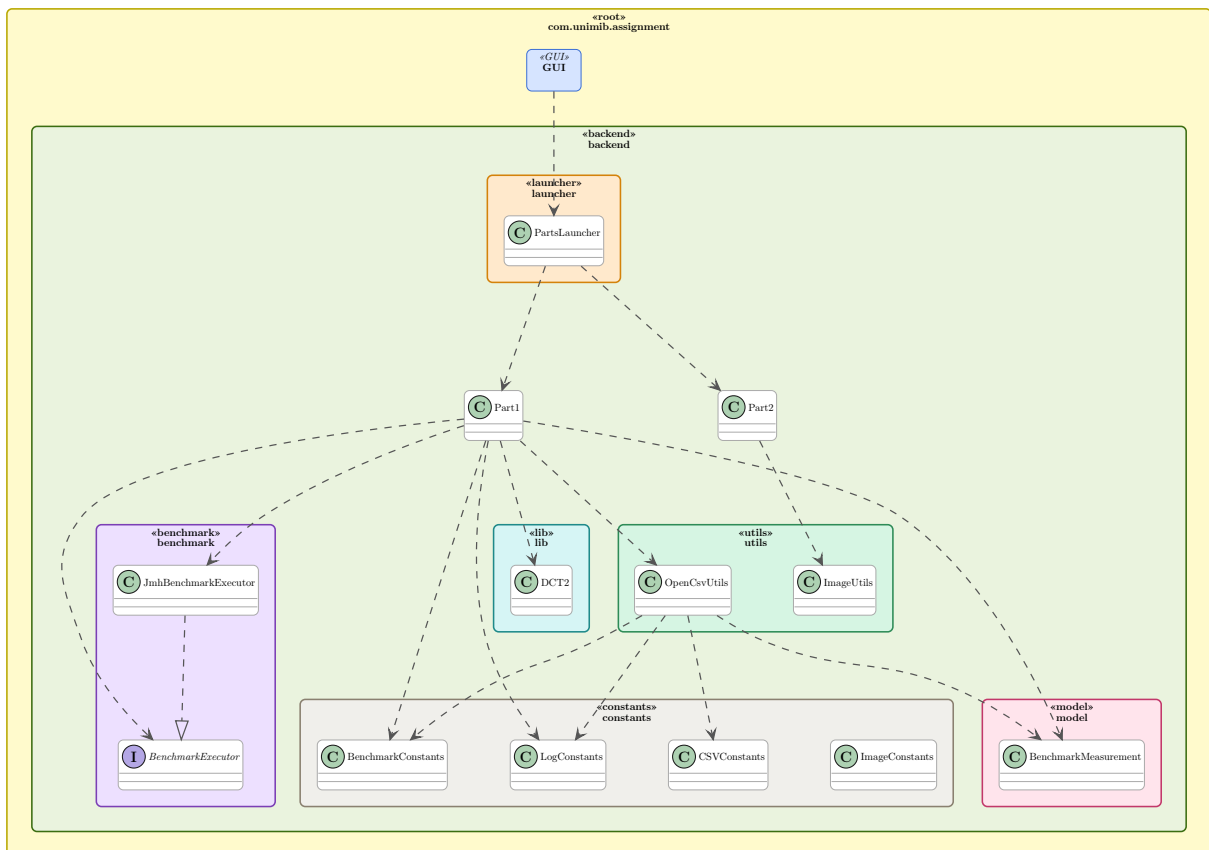


Figura 3.1: Struttura del progetto

3.1 Implementazione della DCT2

Il package `com.unimib.assignment.backend.lib` contiene il metodo `DCTII` che implementa la DCT2. Questo metodo è utilizzato sia per il calcolo della DCT2 (forward) che per la IDCT2 (inverse), in base al metodo che lo richiama. Dato che il calcolo della DCT2 e della IDCT2 differisce solo per la matrice dei coefficienti, abbiamo deciso di implementare un unico metodo che accetta come parametri le matrici dei coefficienti per la DCT2 e la IDCT2, evitando così la duplicazione del codice. L'implementazione della DCT2 è mostrato nel listing 3.1, mentre i metodi che calcolano la DCT2 e la sua inversa sono mostrati rispettivamente nei listing 3.2 e 3.3.

```
1 private SimpleMatrix DCTII(SimpleMatrix signal, SimpleMatrix Dn, SimpleMatrix Dm){
2     // Create a copy of the input signal to avoid modifying the caller's matrix
3     SimpleMatrix result = signal.copy();
4
5     // First pass: Apply column-wise transformation (multiply each column by Dn)
6     // This corresponds to transforming along the column dimension
7     for (int i = 0; i < result.getNumCols(); i++) {
8         // Extract the i-th column as a vector(false flag is used to extract column)
9         SimpleMatrix col = result.extractVector(false, i);
10        // Apply the transform: Dn * column
11        SimpleMatrix transformed = Dn.mult(col);
12        // Insert the transformed column back into the result matrix
13        result.insertIntoThis(0, i, transformed);
14    }
15
16    // Second pass: Apply row-wise transformation (multiply each row by Dm)
17    // This corresponds to transforming along the row dimension
18    for (int i = 0; i < result.getNumRows(); i++) {
19        // Extract the i-th row as a vector(true flag is used to extract row)
20        SimpleMatrix row = result.extractVector(true, i);
21        // Apply the transform: (Dm * (row)^T)^T
22        SimpleMatrix transformed = Dm.mult(row.transpose()).transpose();
23        // Insert the transformed row back into the result matrix
24        result.insertIntoThis(i, 0, transformed);
25    }
26
27    return result;
28 }
```

Listing 3.1: Algoritmo DCTII(DCT2) implementato nel package lib

```
1 public SimpleMatrix forward(SimpleMatrix signal) {
2     // Compute DCT basis matrices for the row and column dimensions
3     SimpleMatrix Dn = computeDMatrix(signal.getNumRows());
4     SimpleMatrix Dm = computeDMatrix(signal.getNumCols());
5     // Apply the forward DCT transformation
6     return DCTII(signal, Dn, Dm);
7 }
```

Listing 3.2: Algoritmo forward implementato nel package lib

```
1 public SimpleMatrix inverse(SimpleMatrix signal) {
2     // Compute DCT basis matrices for the row and column dimensions
3     SimpleMatrix Dn = computeDMatrix(signal.getNumRows());
4     SimpleMatrix Dm = computeDMatrix(signal.getNumCols());
5     // Apply the inverse DCT transformation
6     // (using transposed basis matrices to perform the inverse operation)
7     return DCTII(signal, Dn.transpose(), Dm.transpose());
8 }
```

Listing 3.3: Algoritmo inverse implementato nel package lib

3.2 Gestione dell'esecuzione

Il package `com.unimib.assignment.backend.launcher` contiene la classe `PartsLauncher`, che espone alla UI due metodi:

`launchAndHandlePart1`

Si occupa di invocare la classe `Part1`, responsabile della gestione del processo di esecuzione dei benchmark e della memorizzazione dei risultati.

Come prima operazione, richiama il metodo `randomMatrix` per generare le matrici casuali da utilizzare nei benchmark. Tali matrici vengono create prima dell'esecuzione dei test e condivise tra il benchmark della nostra implementazione della DCT2 e quello della libreria, così da garantire risultati confrontabili. Le dimensioni delle matrici sono memorizzate nell'array di interi `BENCHMARK_BLOCK_SIZES`.

Per l'esecuzione dei benchmark, la classe `Part1` si occupa di:

1. Invocare la classe `JmhBenchmarkExecutor`, responsabile della configurazione e dell'esecuzione dei benchmark tramite JMH. Tale classe gestisce le iterazioni di warmup (`DEFAULT_WARMUP_ITERATIONS = 3`) e quelle di misurazione (`DEFAULT_MEASUREMENT_ITERATIONS = 5`), salvando i risultati in una struttura dati apposita (record). I tempi sono memorizzati in secondi.
2. Salvare i risultati contenuti nel record in un file CSV utilizzando la classe `OpenCsvUtils`, che si appoggia alla libreria open-source `OpenCSV` [6].
3. Gestire eventuali richieste di terminazione forzata del benchmark provenienti dalla UI, interrompendone l'esecuzione.

`launchPart2`

Una volta scelta un'immagine, questa classe si occupa di inviare alla UI la sua versione compressa e di salvarla in formato BMP. La compressione dell'immagine avviene invocando la classe `Part2`, la quale si occupa di eseguire la compressione delle immagini utilizzando la trasformata DCT2 fornita dalla libreria `JTransform`. La classe `Part2` contiene i seguenti metodi:

- `BufferedImage compress(Pair<String, BufferedImage> imageInfo, int F, int d)`: esegue l'intera pipeline di compressione dell'immagine:
 1. Converte l'immagine in input in una matrice di `double`.
 2. Ritaglia la matrice in modo che le sue dimensioni siano multipli di `F`.
 3. Per ogni blocco di dimensione $F \times F$, richiama il metodo `compressBlock` per effettuare la compressione.
 4. Converte la matrice risultante in una `BufferedImage`.
 5. Salva l'immagine compressa nella cartella `output` e successivamente la inoltra alla UI.
- `void compressBlock(double[][] signal, int i, int j, int F, int d)`: comprime un blocco dell'immagine di dimensione $F \times F$:
 1. Copia il blocco in una matrice di `double`.
 2. Applica la DCT2 al blocco utilizzando il metodo `forward(matrix, scale)` dell'oggetto `DoubleDCT_2D` della libreria `JTransform`, impostando `scale = true`. In questo modo viene utilizzata una normalizzazione ortonormale, coerente con quella adottata nella nostra implementazione della DCT2.
 3. Elimina le componenti ad alta frequenza in base al parametro `d`.
 4. Applica la trasformata inversa IDCT2 mediante il metodo `inverse(matrix, scale)`, con `scale = true`.
 5. Richiama il metodo `shiftBlockBy255` per riportare i valori del blocco nell'intervallo `[0, 255]`.
 6. Copia il blocco compresso nella matrice dell'immagine risultante.
- `void shiftBlockBy255(double[][] matrix)`: effettua uno shift dei valori nell'intervallo `[0, 255]`, riportandoli nel dominio delle immagini BMP.

I metodi di conversione tra `BufferedImage` e matrici di `double` e il salvataggio in formato BMP sono implementati nella classe `ImageUtils`.

3.2.1 GUI

Il package `com.unimib.assignment.GUI` offre 3 schermate principali:

- La schermata iniziale che presenta la scelta tra benchmark e compressione delle immagini, mostrata in figura 3.2 La figura 3.3 mostra invece la schermata iniziale una volta scelta l'opzione di benchmark, che presenta un pulsante per interromperne l'esecuzione.
- Una schermata per la selezione dell'immagine da comprimere, che mostra un'anteprima dell'immagine selezionata e, una volta effettuata, il risultato della compressione. Tale schermata è mostrata in figura 3.4
- Una schermata per la scelta dei parametri di compressione, mostrata in figura 3.5.

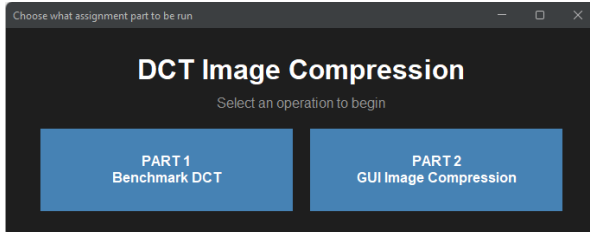


Figura 3.2: Schermata iniziale per la scelta tra benchmark e compressione delle immagini

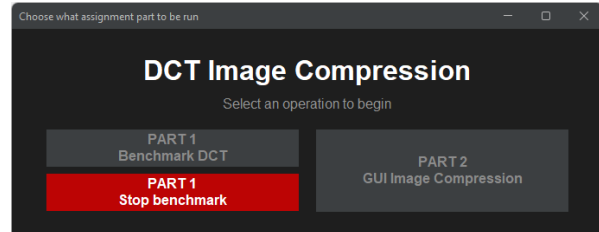


Figura 3.3: Schermata iniziale una volta scelta l'opzione di benchmark



Figura 3.4: Schermata di compressione delle immagini

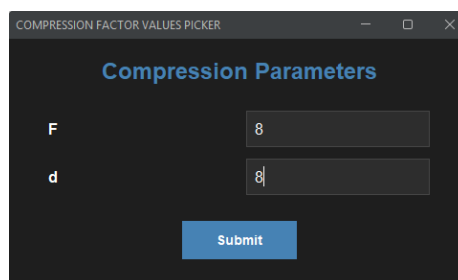


Figura 3.5: Schermata d'inserimento dei parametri di compressione

Bibliografia

- [1] P. Wendykier, *JTransforms*, ver. 3.1, Open-source multithreaded FFT library written in pure Java, 2015. indirizzo: <https://github.com/wendykierp/JTransforms>
- [2] P. Abeles, *Efficient Java Matrix Library (EJML)*, ver. 0.44.0, Fast and easy-to-use linear algebra library written in Java, 2025. indirizzo: <https://github.com/lessthanoptimal/ejml>
- [3] OpenJDK, *JMH: Java Microbenchmark Harness*, Accessed: 2026-05-25, 2026. indirizzo: <https://github.com/openjdk/jmh>
- [4] IBM, *How the JIT Compiler Optimizes Code*, Accessed: 2026-05-25, 2026. indirizzo: <https://www.ibm.com/docs/en/sdk-java-technology/8?topic=compiler-how-jit-optimizes-code>
- [5] P. Wendykier, *DoubleDCT_2D (JTransforms API Documentation)*, https://javadoc.io/doc/com.github.wendykierp/JTransforms/latest/org/jtransforms/dct/DoubleDCT_2D.html, Accessed: 2026-05-27, 2026.
- [6] OpenCSV Project Contributors, *OpenCSV: A Simple CSV Parser for Java*, <https://opencsv.sourceforge.net/>, Version 5.12.0, accessed 2026-05-30, 2025.